



Servicio Nacional de Aprendizaje – SENA

Tecnólogo en Análisis y Desarrollo de Software
Ficha: 3134555

**verificación de procedimientos para la definición de componentes frontend de la aplicación
(listas de chequeo)
(GA7-220501096-AA4-EV02)**

APRENDIZ
Johan Mauricio Sánchez Gaviria

INSTRUCTOR

FEBRERO 2026

Introducción

Este documento define los componentes del frontend en React para el sistema web ALUMIPRO. La definición se hace bajo un enfoque de arquitectura limpia (separación por capas UI/servicios) y con justificación técnica de cada componente en función de su responsabilidad y reutilización.

Objetivo

Diseñar el catálogo de componentes React del proyecto ALUMIPRO, justificando su existencia para asegurar mantenibilidad, reutilización, escalabilidad y coherencia visual, preparando el frontend para consumo de API.

Arquitectura frontend propuesta

Estructura por capas:

- pages/: pantallas (rutas) del sistema.
- components/: componentes reutilizables (UI y composición).
- services/: capa de acceso a datos (HTTP) desacoplada de la UI.
- hooks/: lógica reutilizable (fetch, formularios, etc.).
- types/: tipos/DTOs del dominio (Cliente, Producto, Venta).
- utils/: validaciones, formateadores.

Esta organización evita mezclar lógica de negocio con presentación y facilita pruebas y migraciones.

Catálogo de componentes y justificación

• AppLayout

- Descripción: Contenedor principal con estructura base (header/nav/footer).
- Justificación: Centraliza layout global y evita duplicación de estructura en cada página.

• Navbar

- Descripción: Menú de navegación (Clientes, Productos, Ventas).
- Justificación: Mejora UX y mantiene navegación consistente; facilita extensión futura.

- **PageTitle**

- Descripción: Encabezado estándar por página (título + acciones).
- Justificación: Uniforma estilo y reduce código repetido en cada vista.

- **Alert**

- Descripción: Mensajes de error/éxito/advertencia.
- Justificación: Estandariza comunicación al usuario y el manejo de estados (error/ok).

- **LoadingSpinner**

- Descripción: Indicador de carga.
- Justificación: Mejora experiencia en operaciones async; evita pantallas ‘congeladas’.

- **DataTable**

- Descripción: Tabla genérica parametrizable (columnas + data + acciones).
- Justificación: Reutilizable para listados (clientes/productos) con acciones estándar.

- **ConfirmDialog**

- Descripción: Confirmación antes de eliminar.
- Justificación: Reduce errores de usuario; patrón común en operaciones destructivas.

- **ClienteListPage**

- Descripción: Pantalla de listado de clientes.
- Justificación: Representa caso de uso: consultar clientes; se apoya en DataTable.

- **ProductoListPage**

- Descripción: Pantalla de listado de productos.
- Justificación: Caso de uso: consultar productos y acceder a CRUD.

- **ProductoForm**

- Descripción: Formulario de creación/edición de producto.
- Justificación: Reutilización: mismo formulario para crear/editar con validaciones.

- **VentaCreatePage**

- Descripción: Pantalla para registrar venta (cliente + items).
- Justificación: Caso crítico del negocio: venta transaccional con validación de stock.

- **VentaItemRow**

- Descripción: Fila de ítem de venta (producto + cantidad).
- Justificación: Permite crecer a N items (dinámico) sin duplicar campos.

- **NotFoundPage**

- Descripción: Página 404.

- Justificación: Manejo estándar de rutas inválidas; mejora robustez.

Notas de integración con backend

El frontend se implementará preparado para consumo de servicios HTTP (AA5). En AA4 se deja una capa services con baseURL configurable, permitiendo:

- Ejecutar en desarrollo con proxy hacia el backend.
- Generar build estático y servirlo desde Spring Boot (opcional).

La lógica crítica de transacciones permanece en el backend; el frontend solo valida formatos y rangos básicos.